RockyBot AI: News & Financial Research Engine

Complete Technical Architecture, RAG Pipeline & Developer Guide for Freshers

1. Executive Summary & Main Goal

Financial analysts, equity researchers, and investors regularly need to review breaking news, earnings announcements, and market commentary across multiple web sources. Manually reading through hundreds of lines of text across various tabs is tedious, and generic LLMs (like standard ChatGPT) hallucinate or lack access to live URL content.

The Goal of RockyBot: To provide an automated AI Research Assistant where users can enter **one or more article URLs**, index their content in a high-speed local vector database (FAISS), and ask targeted analytical questions. RockyBot returns concise, factual answers backed by **exact, clickable source citations**, eliminating hallucinations completely.

2. Core Concepts: Are We Using RAG, LangChain, Langfuse, Vector DB?

Technology / Concept	Used in Project?	What It Does in RockyBot
RAG (Retrieval-Augmented Generation)	**YES (Core)**	Instead of letting the LLM guess, we first retrieve the top matching text chunks from the vector database and provide them as context in the prompt.
LangChain	**YES (Core Framework)**	Orchestrates the entire RAG pipeline: WebBaseLoader (web scraping), RecursiveCharacterTextSplitter (chunking), GoogleGenerativeAIEmbeddings, and RetrievalQAWithSourcesChain.
Vector DB (FAISS)	**YES (Local Store)**	Facebook AI Similarity Search (FAISS) stores 1536-dimensional semantic vectors on disk and performs sub-millisecond similarity search to find relevant passages.
Langfuse	**NO**	Langfuse is an external LLM telemetry/observability platform (similar to LangSmith). It is NOT used or required in this project.

3. Directory & File Structure: What Does Each Part Do?

Folder / File	Purpose & Working Mechanism
server.py	Production **FastAPI** backend server. Implements REST endpoints (/api/process-urls, /api/query, /api/status, /api/reset), document cleaning, anti-bot validation, rate-limited Gemini embeddings, and serves static files.
static/ (index.html, index.css, app.js)	**Frontend Web Application**. Pure Vanilla HTML/CSS/JS with sleek dark-mode glassmorphism, responsive grid, dynamic 1+ URL inputs, progress stepper, prompt chips, and citation cards.

run.py	Convenient one-click launcher script. Starts the Uvicorn server and automatically opens the user's browser at http://localhost:8000.
main.py	The legacy **Streamlit** interface. Updated to support dynamic single or multi-URL inputs with session state, but runs as a monolithic Streamlit app.
faiss_store_gemini/	Local storage folder for the active FAISS index: contains index.faiss (vector index), index.pkl (document metadata), and sources_metadata.json (URL domains and chunk counts).
notebooks/	Jupyter notebooks for experimentation and learning (faiss_tutorial.ipynb, retrieval.ipynb, text_loaders_splitters.ipynb, sample text files).
__pycache__/	Standard Python internal cache directory. Python compiles human-readable .py files into .pyc bytecode to speed up subsequent imports. It can be safely deleted; Python regenerates it automatically.
.env & **.env.example**	Stores private environment variables (GOOGLE_API_KEY). Never committed to public version control.
requirements.txt	Declares Python library dependencies: fastapi, uvicorn, langchain, langchain-google-genai, faiss-cpu, python-dotenv, etc.
sample_article_urls.txt	Curated list of verified free URLs (MoneyControl, CNBC, BBC, Yahoo Finance, Wikipedia) ready for immediate testing.

4. Frontend Tech, Backend Tech & Network Ports

The project uses a clean client-server architecture decoupled into modern web layers:

Layer	Technology Used	Port / URL	Key Role
Frontend	HTML5, Vanilla CSS3, Vanilla ES6+ JavaScript	Port **8000** (Served by FastAPI)	Dynamic URL builder, status check, stepper animations, prompt chips, citation shelf.
Backend API	Python 3.11, FastAPI, Uvicorn (ASGI)	**http://127.0.0.1:8000**	Processes HTTP requests, validates inputs with Pydantic, orchestrates LangChain RAG pipeline.
Legacy UI	Streamlit (Python)	Port **8501** (Optional)	Fallback monolithic data science interface.
LLM & Embeddings	Google Gemini API (Cloud)	HTTPS (Port 443 Outbound)	gemini-embedding-2 (1536 dimensions) & gemini-flash-lite-latest (synthesis).

5. How Gemini API is Handled & Integrated

Two Gemini Models are Employed:

- **GoogleGenerativeAIEmbeddings:** Model gemini-embedding-2 produces 1536-dimensional semantic vectors for text chunks.
- **ChatGoogleGenerativeAI:** Model gemini-flash-lite-latest (temperature 0.2) delivers sub-second, highly factual synthesis without creative hallucinations.

Solving the 429 Quota Rate Limit (ThrottledGoogleEmbeddings):
On Google's Free Tier, embedding requests are limited to 100 requests per minute (RPM). Ingesting large pages can easily trigger HTTP 429. We built a specialized wrapper ThrottledGoogleEmbeddings that:

1. Batches chunks into micro-groups of 15 with a 0.6s breather interval.
2. Automatically catches 429 RESOURCE_EXHAUSTED errors and extracts the retryDelay from the response.
3. Automatically pauses and retries up to 3 times in the background without user intervention or crashing.
4. Trims oversized documents (max 35,000 characters) and caps chunk batches to 40 per run.

6. End-to-End Workflow: Ingestion to Q&A;

Step 1 - User Inputs URLs: User enters 1 or more URLs in the frontend (or clicks a sample button). Only 1 URL is needed to proceed.
Step 2 - Scrape & Validate: WebBaseLoader downloads the HTML using standard browser headers. An automated validator screens out paywalls, CAPTCHAs, or empty content.
Step 3 - Text Chunking: RecursiveCharacterTextSplitter divides text into 1500-character segments with 150-character overlap using semantic separators.
Step 4 - Vector Indexing: Chunks are converted to embeddings via Gemini and indexed into FAISS on disk (faiss_store_gemini/).
Step 5 - Question Submission: User types a query. FAISS performs similarity search and extracts the top 4 most relevant text passages.
Step 6 - LLM Synthesis & Citations: The passages and question are passed to Gemini Flash Lite via a strict prompt

requiring source attribution. The final answer and verified links are displayed with a one-click copy button.

7. Quick Interview Cheatsheet for Freshers

Q: What is RAG and why not just prompt the LLM directly?

A: RAG injects retrieved real-world documents into the prompt at runtime. Standard LLMs hallucinate or lack access to today's news articles. RAG ensures 100% factual accuracy and source traceability.

Q: Why use FAISS instead of a cloud database like Pinecone?

A: FAISS runs locally in-process with zero network latency, zero operational cost, and direct disk persistence, making it ideal for self-contained desktop and web tools.

Q: Why decouple into FastAPI and pure HTML/JS instead of Streamlit?

A: Streamlit re-executes the entire script on every user interaction, causing latency and layout rigidity. A decoupled FastAPI + Vanilla JS frontend enables smooth animations, asynchronous requests, and full design freedom.